

МИНОБРНАУКИ РОССИИ
ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО УЧЕБНОЙ ОЗНАКОМИТЕЛЬНОЙ
ПРАКТИКЕ

Студент
гр. БИН-24-2

_____ Е. Е. Попов

Руководитель,
д-р. техн. наук

_____ В. М. Гриняк

Консультант,
ассистент

_____ М. В. Водяницкий

Владивосток 2026

МИНОБРНАУКИ РОССИИ
ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ
на учебную ознакомительную практику

Студенту: гр. БИН-24-2 Попову Е. Е.

Срок сдачи работы: 15.07.2026

Задание 1. Анализ поставленной задачи, раскрытие цели и задачи разработки приложения, раскрытие сущности поставленной задачи.

Задание 2. Сбор и анализ информации, определение технических требований к алгоритмам и кроссплатформенной графике для отображения структур данных, определение источников необходимой информации, поиск и систематизирование данные о механизмах работы выбранных структур.

Задание 3. Разработка решения, формулирование выводов по итогам разработки, тестирование корректности работы.

Задание 4. Оформить отчет по практике в соответствии с СК-СТО-ТР-04-1.005-2015 «Требования к оформлению текстовой части выпускных квалификационных работ, курсовых работ (проектов), рефератов, контрольных работ, отчётов по практикам, лабораторным работам».

Руководитель практики	_____	В. М. Гриняк
Консультант практики	_____	М. В. Водяницкий
Задание получил:	_____	Е. Е. Попов

КАЛЕНДАРНЫЙ ПЛАН-ГРАФИК (ДНЕВНИК)
ПРОХОЖДЕНИЯ УЧЕБНОЙ ОЗНАКОМИТЕЛЬНОЙ ПРАКТИКИ
СТУДЕНТА ВВГУ

Студент Е. Е. Попов направляется для прохождения учебной ознакомительной практики с 15.06.26 по 18.07.26

№ п/п	Содержание выполняемых работ по программе	Сроки выполнения		Заключение и оценка руководителя или консультанта	Подпись руководителя или консультанта
		Начало	Окончание		
1	Анализ требований, определение цели и задач разработки. Выбор инструментов реализации	15.06.2026	18.06.2026		
2	Проектирование логики приложения, реализация визуализатора, проведение тестирования работы алгоритмов	18.06.2026	28.06.2026		
3	Систематизация результатов, формулирование выводов по итогам разработки	28.06.2026	02.07.2026		
4	Оформление отчета	02.07.2026	15.07.2026		

Согласовано:

Студент практикант

_____ Е. Е. Попов
подпись

дата

Руководитель от кафедры

_____ В. М. Гриняк
подпись

Консультант от кафедры

_____ М. В. Водяницкий
подпись

Содержание

Введение	3
1 Теоретические основы структур данных	4
1.1 Понятие абстрактного типа данных и линейной структуры	4
1.2 Стек (LIFO).....	4
1.3 Очередь (FIFO).....	6
1.4 Двоичное дерево поиска (BST).....	7
2 Анализ требований и проектирование архитектуры приложения	8
2.1 Постановка задачи и функциональные требования	8
2.2 Нефункциональные требования и требование кроссплатформенности.....	8
2.3 Архитектура приложения.....	10
2.4 Выбор средств тестирования и контроля корректности	12
3 Реализация и тестирование визуализатора структур данных.....	13
3.1 Общая структура программы и точка входа.....	13
3.2 Панель управления стеком и отображение состояния	13
3.3 Механизм Undo для стека	15
3.4 Реализация и визуализация очереди	16
3.5 Реализация и визуализация двоичного дерева поиска	19
3.6 Тестирование корректности реализованных алгоритмов	23
3.7 Проверка кроссплатформенной работоспособности	24
Заключение.....	25
Список использованных источников	26
Блок-схема алгоритма двоичного дерева поиска (BST).....	28

Введение

В современной индустрии разработки программного обеспечения и компьютерных наук фундаментальное понимание базовых структур данных и алгоритмов является основой для подготовки квалифицированных ИТ-специалистов. Такие структуры, как стек, очередь и двоичное дерево поиска (BST), представляют собой классические инструменты организации данных, используемые при решении широкого спектра задач – от управления системной памятью до построения сложных поисковых индексов. Однако для начинающих разработчиков и студентов теоретическое освоение механизмов их работы часто вызывает затруднения из-за высокой абстрактности концепций.

Визуальное моделирование и интерактивная демонстрация работы алгоритмов в реальном времени позволяют сделать процесс обучения наглядным и эффективным. Особое значение приобретает разработка инструментов, способных не просто отображать конечный результат, но и демонстрировать пошаговые изменения состояния структур данных после каждой операции, а также предоставлять пользователю возможность отката изменений. Использование языка программирования Python позволяет эффективно реализовать логику алгоритмов и обеспечить кроссплатформенность программного решения, что существенно расширяет доступность и универсальность разработанного продукта.

Целью работы является разработка кроссплатформенного программного обеспечения для интерактивной визуализации стека, очереди и двоичного дерева поиска с поддержкой пошагового выполнения операций, динамического отображения состояний структур и ведения лога с функцией отмены действий.

Задачи исследования:

- 1) изучить теоретические основы организации и принципы работы структур данных;
- 2) выбрать оптимальные инструменты разработки на языке Python и спроектировать архитектуру приложения;
- 3) разработать графический интерфейс пользователя и реализовать программный код визуализатора;
- 4) протестировать корректность работы реализованных алгоритмов;
- 5) подготовить отчёт, включая введение, отчётные части и заключение.

Таким образом, разработанное приложение не только служит учебным инструментом для наглядного освоения фундаментальных структур данных, но и предоставляет удобные средства контроля выполняемых действий. Структура и оформление настоящего отчёта соответствуют требованиям локального стандарта вуза [1].

1 Теоретические основы структур данных

1.1 Понятие абстрактного типа данных и линейной структуры

В современной теории программирования абстрактный тип данных представляет собой универсальную математическую модель данных совместно с набором базовых операций, определённых над этой моделью, причём принципиально без указания конкретного способа их внутренней программной реализации. Стек и очередь традиционно относятся к категории линейных динамических структур данных, для которых характерно то, что все элементы в них логически упорядочены друг за другом, а непосредственный доступ к произвольному элементу, минуя предшествующие, в общем случае категорически не предусмотрен функциональным интерфейсом структуры. Это обстоятельство принципиально отличает их от классического массива, в котором всегда гарантированно обеспечивается прямой доступ по индексу за константное время. Указанное фундаментальное отличие напрямую определяет и различие в характере решаемых практических задач: массив абсолютно предпочтителен там, где заранее известен конечный размер данных и требуется произвольный доступ к элементам, тогда как стек и очередь активно применяются в ситуациях, где важен не произвольный доступ, а строго определённый порядок обработки поступающих элементов – соответственно LIFO или FIFO, – что и обуславливает их широкое использование во внутренних механизмах различных языков программирования и операционных систем.

Будучи абстрактными типами данных, стек и очередь не предписывают конкретного способа хранения элементов: на практике они могут быть реализованы как поверх массива, так и на основе связного списка, а в языках высокого уровня – с использованием стандартных библиотек. С точки зрения пользователя, однако, всегда доступен лишь строго определённый набор операций (`push/pop` или `enqueue/dequeue` вместе с проверкой на пустоту), гарантирующий соблюдение дисциплины LIFO или FIFO независимо от внутреннего устройства. Подобное отделение интерфейса от реализации – один из центральных принципов модульного программирования, позволяющий заменять одну реализацию другой без модификации клиентского кода и защищающий целостность структуры данных.

1.2 Стек (LIFO)

Стек – это структура данных, организованная по принципу «последним пришёл – первым вышел» (Last In, First Out, LIFO). Над стеком определены две основные операции изменения состояния и одна операция чтения:

- 1) `push` (добавление) – помещает новый элемент на вершину стека;
- 2) `pop` (удаление) – извлекает и удаляет элемент с вершины стека;

- 3) peek, или top (просмотр) – возвращает значение верхнего элемента без его удаления из структуры.

При программной реализации на основе динамического массива (как в настоящей работе) все три операции выполняются за время $O(1)$ в среднем, поскольку добавление и удаление производятся только с одного конца последовательности – её «хвоста», что не требует сдвига остальных элементов. Классическими примерами применения стека являются: организация стека вызовов функций в большинстве языков программирования, проверка сбалансированности скобочных последовательностей, реализация алгоритмов обхода графов в глубину (DFS) и, что наиболее существенно для данной работы, реализация механизма отмены последнего действия (Undo) в интерактивных приложениях [2].

Блок-схема алгоритма работы стека (LIFO), выполненная в соответствии со стандартными условными обозначениями [3], приведена на рисунке 1. Она наглядно демонстрирует, что новая запись всегда помещается на вершину, а извлечение происходит с той же позиции, что обеспечивает дисциплину «последним пришёл – первым вышел».

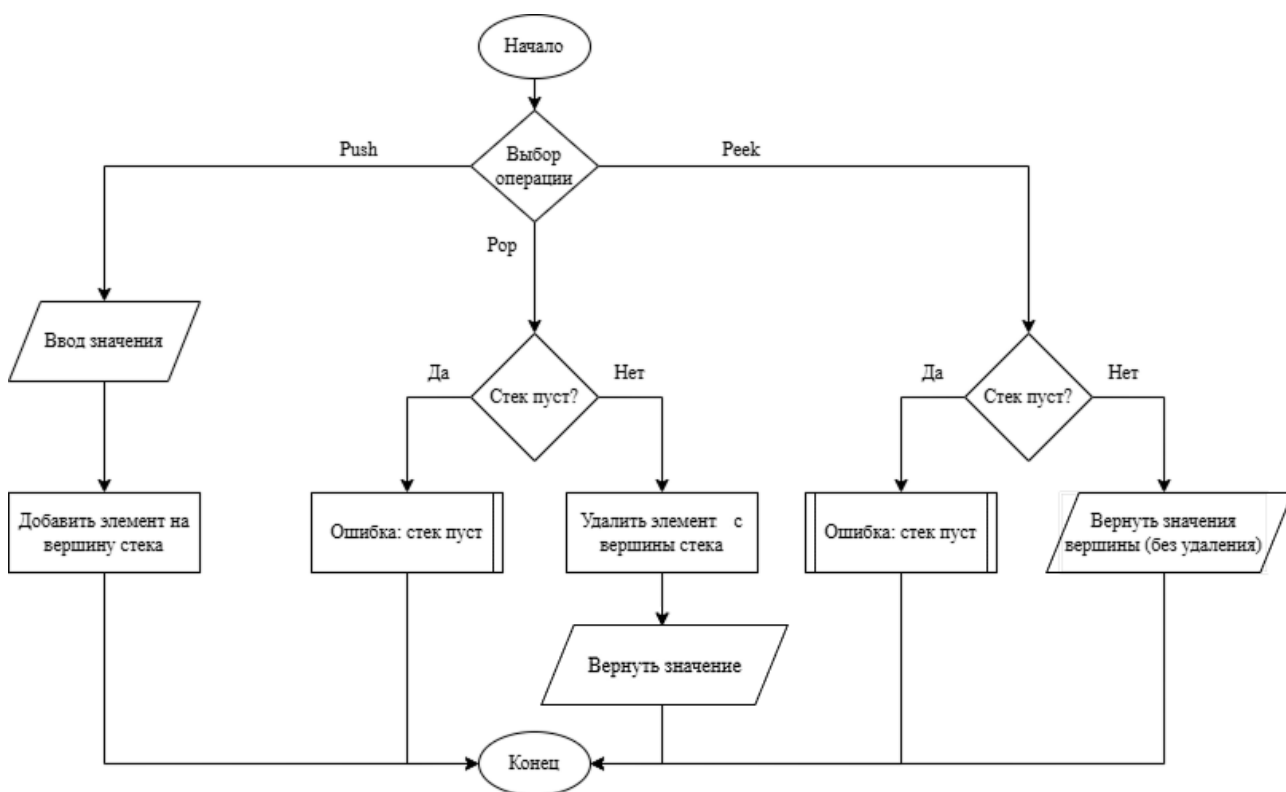


Рисунок 1 – Блок-схема алгоритма работы стека (LIFO)

Таким образом, представленная схема иллюстрирует базовый принцип организации элементов, гарантируя корректное выполнение операций модификации структуры в соответствии с её алгоритмической логикой.

1.3 Очередь (FIFO)

Очередь – структура данных, реализующая принцип «первым пришёл – первым вышел» (First In, First Out, FIFO). Основные операции очереди:

- 1) enqueue (постановка в очередь) – добавляет новый элемент в конец (хвост) очереди;
- 2) dequeue (выход из очереди) – извлекает и удаляет элемент из начала (головы) очереди;
- 3) front (просмотр головы) – возвращает значение первого элемента без его удаления.

В наивной реализации на основе массива операция dequeue, удаляющая элемент с начала последовательности, требует сдвига оставшихся элементов и выполняется за время $O(n)$, где n – текущее число элементов в очереди. Для целей демонстрационной визуализации, где число элементов невелико и наглядность важнее асимптотической эффективности, такая реализация является обоснованным проектным решением; в высоконагруженных системах для устранения этого недостатка применяются кольцевой буфер или связный список с двумя указателями (на голову и хвост), обеспечивающие амортизированное время $O(1)$ для обеих операций [4]. Очереди широко используются при моделировании систем обслуживания заявок, в планировщиках задач операционных систем, в реализации алгоритма обхода графа в ширину (BFS) [2] и в буферизации данных при асинхронном вводе-выводе. Блок-схема алгоритма работы очереди (FIFO) приведена на рисунке 2.

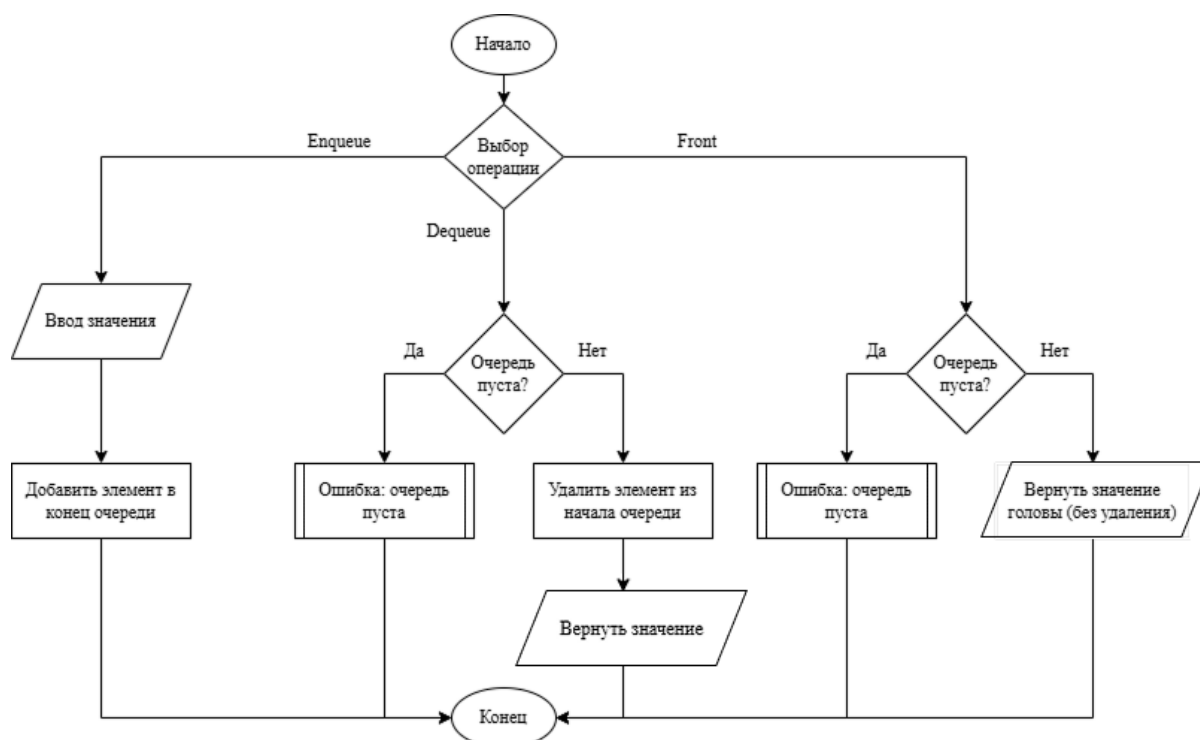


Рисунок 2 – Блок-схема алгоритма работы очереди (FIFO)

Данная визуальная модель наглядно демонстрирует взаимосвязь узлов и распределение данных, что позволяет однозначно интерпретировать изменения, происходящие при выполнении базовых операций.

1.4 Двоичное дерево поиска (BST)

Двоичное дерево поиска (Binary Search Tree, BST) – это нелинейная иерархическая структура данных, в которой каждый узел содержит не более двух потомков (левого и правого) и удовлетворяет основному инварианту: значения всех узлов левого поддерева строго меньше значения текущего узла, а значения всех узлов правого поддерева строго больше его значения. Этот инвариант рекурсивно выполняется для каждого поддерева целиком, что и обеспечивает возможность поиска.

Операции (BST):

- а) операция вставки – вставка нового значения в BST выполняется путём рекурсивного (либо итеративного) спуска от корня: на каждом шаге сравнивается вставляемое значение с значением текущего узла, и движение продолжается в левое поддерево, если вставляемое значение меньше, либо в правое – если больше; при достижении пустой ссылки (отсутствующего узла) на этом месте создаётся новый узел;
- б) операция поиска – поиск значения в BST повторяет логику вставки: начиная с корня, на каждом шаге искомое значение сравнивается со значением текущего узла; при равенстве поиск завершается успехом, при несовпадении – продолжается в соответствующее поддерево до тех пор, пока либо не будет найден искомый узел, либо не будет достигнута пустая ссылка, что означает отсутствие значения в дереве;
- в) операция удаления – наиболее алгоритмически нетривиальная операция среди трёх рассмотренных структур, поскольку требует сохранения основного инварианта дерева после удаления. Различают три случая:
 - 1) удаляемый узел является листом (не имеет потомков) – узел удаляется без дополнительных действий;
 - 2) удаляемый узел имеет одного потомка – узел заменяется этим единственным потомком;
 - 3) удаляемый узел имеет двух потомков – узел заменяется значением минимального элемента своего правого либо левого поддерева, после чего этот элемент-донор рекурсивно удаляется из правого поддерева, где он гарантированно представляет один из первых двух случаев.

Временная сложность всех трёх операций – $O(h)$, где h – высота дерева. В сбалансированном дереве она составляет $O(\log n)$, что гарантирует логарифмическую эффективность, однако при строго упорядоченной вставке дерево вырождается в линейный список, и сложность возрастает до $O(n)$. Для устранения этого недостатка применяются самобалансирующиеся BST [2], но их рассмотрение выходит за рамки данной работы. Алгоритм работы двоичного дерева поиска подробно показан на рисунке А.1 (приложение А).

2 Анализ требований и проектирование архитектуры приложения

2.1 Постановка задачи и функциональные требования

Согласно индивидуальному заданию на практику, разрабатываемое приложение должно реализовывать визуальные модели трёх структур данных – стека, очереди и двоичного дерева поиска – таким образом, чтобы пользователь мог выполнять основные операции каждой структуры в пошаговом режиме и видеть графическое отображение состояния структуры, изменившееся в результате каждой выполненной операции. Дополнительным функциональным требованием является ведение журнала (лога) выполненных операций с возможностью отмены последнего действия (Undo). На основании задания сформулирован следующий перечень функциональных требований к приложению:

- 1) приложение должно предоставлять отдельный пользовательский интерфейс для каждой из трёх структур данных, переключаемый без перезапуска программы;
- 2) для стека должны быть реализованы операции `push` (добавление значения), `pop` (удаление верхнего значения) и `peek` (просмотр верхнего значения без удаления);
- 3) для очереди должны быть реализованы операции `enqueue` (постановка значения в конец), `dequeue` (извлечение значения из начала) и `front` (просмотр первого значения без удаления);
- 4) для двоичного дерева поиска должны быть реализованы операции `insert` (вставка значения), `delete` (удаление значения) и `search` (поиск значения с визуализацией пройденного пути);
- 5) после выполнения каждой операции содержимое структуры должно перерисовываться на графическом холсте, отражая её текущее состояние;
- 6) каждая операция должна сопровождаться записью в общий текстовый лог с указанием времени выполнения, типа структуры и сути произошедшего изменения;
- 7) для каждой структуры должна быть реализована функция `Undo`, отменяющая последнее выполненное над ней изменяющее действие и возвращающая структуру к предыдущему зафиксированному состоянию.

Перечисленные требования образуют функциональный каркас приложения и служат основой для проектирования его архитектуры и пользовательского интерфейса.

2.2 Нефункциональные требования и требование кроссплатформенности

Прежде чем переходить к выбору конкретной библиотеки графического интерфейса, необходимо обосновать выбор самого языка программирования, поскольку от этого решения напрямую зависят выполнимость требования кроссплатформенности и сроки разработки в рамках учебной практики. Сравнение Python с наиболее распространёнными альтернативами

для разработки кроссплатформенных настольных приложений –Java, C++ и C# –приведено в таблице 1.

Таблица 1 – Сравнение языков программирования для реализации приложения

Критерий	Python	Java	C++ (Qt)	C#/.NET
Кроссплатформенность «из коробки»	Да, стандартная библиотека включает tkinter для Windows и Linux	Да, через JVM	Да, но требует подключения тяжёлого стороннего фреймворка (Qt, wxWidgets)	Полноценная кроссплатформенность (MAUI/Avalonia) появилась относительно недавно и требует дополнительной настройки
Необходимость установки окружения на целевой машине	Не требуется – типовые дистрибутивы Python уже включают tkinter	Требуется установленная JRE/JDK	Требуется сборка под конкретную ОС и распространение бинарных Qt-библиотек	Требуется .NET runtime либо публикация self-contained сборки
Скорость разработки прототипа	Высокая – динамическая типизация, короткий цикл правка-запуск	Средняя – строгая типизация, больше шаблонного кода	Низкая – ручное управление памятью, сложная сборка	Средняя –привязка к среде Visual Studio
Объём итогового исполняемого файла	Умеренный (PyInstaller)	Умеренный (JAR), но требует JRE	Компактный нативный бинарник	Умеренный/крупный при self-contained публикации

Источники: [5,6,7]

Сравнение показывает, что Python обеспечивает необходимую кроссплатформенность без установки дополнительного окружения на целевой машине, минимальный порог входа и высокую скорость разработки прототипа, что особенно важно в условиях ограниченных сроков учебной практики. Указанные преимущества обусловили выбор Python в качестве языка реализации визуализатора структур данных.

Дополнительно, в рамках выбранного языка Python, было проанализировано несколько альтернативных вариантов реализации графического интерфейса. Итоговое приложение должно одинаково корректно запускаться и работать как в операционных системах семейства Windows, так и в Linux, без необходимости вносить изменения в исходный код или использовать специфичные для конкретной ОС команды и библиотеки.

Для выполнения этого требования при выборе технологического стека были проанализированы несколько альтернативных вариантов реализации графического интерфейса на языке Python, сравнение которых приведено в таблице 2.

Таблица 2 – Сравнение вариантов реализации графического интерфейса

Библиотека	Кроссплатформенность	Необходимость установки
Tkinter	Да, входит в стандартную библиотеку CPython для Windows и Linux	Не требуется (за исключением пакета python3-tk в части дистрибутивов Linux)
PyQt/PySide	Полная, но требует распространения дополнительных бинарных компонентов Qt	Требуется установка стороннего пакета значительного объёма
Веб-интерфейс (Flask/Django + браузер)	Полная (зависит только от наличия браузера)	Требуется веб-сервер и браузер

Источники: [8, 9, 10, 11]

Сравнение, приведённое в таблице 2, показывает, что библиотека Tkinter обладает наилучшим соотношением простоты использования и готовой кроссплатформенности среди рассмотренных вариантов: будучи частью стандартной библиотеки языка Python, она не требует установки дополнительных зависимостей на большинстве дистрибутивов Windows (где интерпретатор Python с официального сайта поставляется с Tkinter по умолчанию) и устанавливается одной командой пакетного менеджера на дистрибутивах Linux, не предусматривающих её предустановку. Именно поэтому Tkinter был выбран в качестве основной технологии реализации графического интерфейса разрабатываемого приложения.

2.3 Архитектура приложения

Для эффективного обеспечения расширяемости и читаемости кода архитектура данного приложения построена по базовому принципу разделения ответственности между логическим (модельным) слоем, отвечающим за хранение данных структур и реализацию алгоритмов, и слоем представления, отвечающим за отображение состояния структур и обработку действий пользователя. Такое разделение полностью соответствует общепринятому архитектурному шаблону MVC (Model-View-Controller) в его упрощённом для современных настольных приложений варианте [12].

В модельном слое выделены четыре класса:

- 1) `stack` – инкапсулирует внутренний список значений и операции `push`, `pop`, `peek`, `is_empty`, а также метод `copy` для создания независимой копии текущего состояния (используется механизмом `Undo`);
- 2) `queue` – аналогичен классу `Stack` по структуре, но реализует операции `enqueue`, `dequeue`, `front` в соответствии с дисциплиной `FIFO`;
- 3) `BSTNode` – представляет один узел двоичного дерева поиска с полями `value`, `left` и `right`;
- 4) `BST` – инкапсулирует ссылку на корневой узел дерева и реализует рекурсивные операции `insert`, `delete`, `search`, а также вспомогательные операции `copy` (глубокое копирование дерева для `Undo`) и хранение последнего пройденного пути поиска (`last_path`) и последнего найденного значения (`last_found`) для целей визуализации.

В слое представления реализован базовый класс `BasePanel`, от которого наследуются три специализированные панели – `StackPanel`, `QueuePanel` и `BSTPanel`, каждая из которых связывает соответствующий класс модели с виджетом `Canvas` для отрисовки и набором кнопок управления. Пример работы одной из таких панелей – вкладки «Дерево (BST)» – показан на рисунке 3.

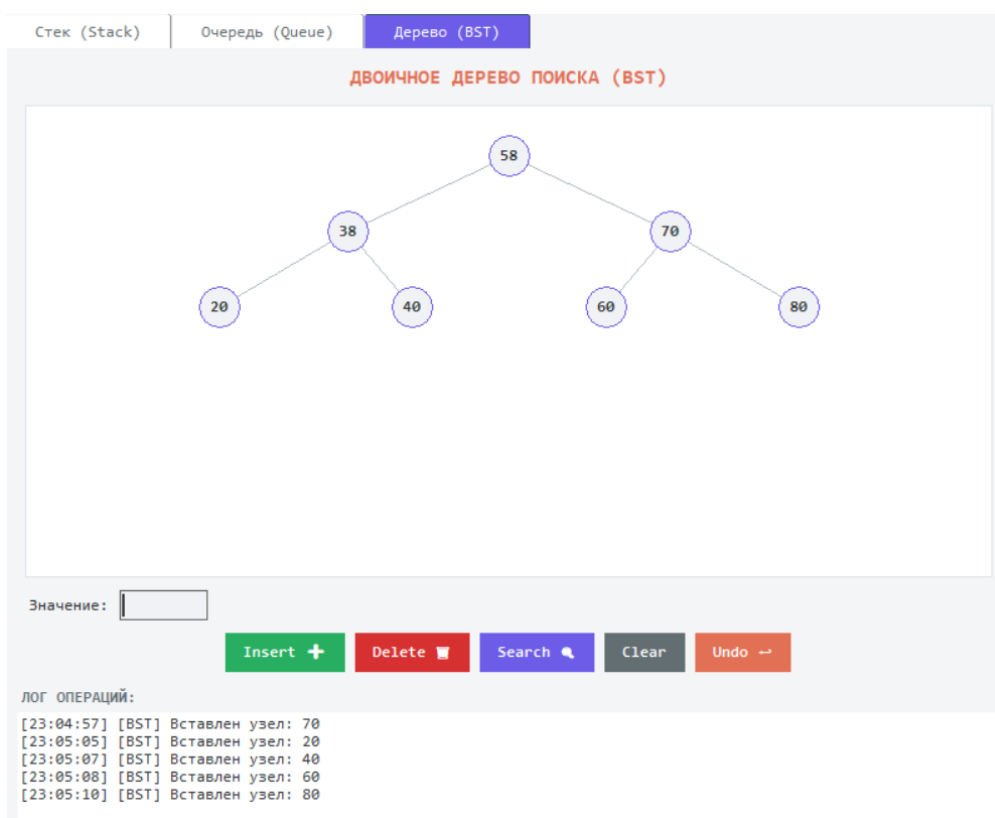


Рисунок 3 – Пример визуализации двоичного дерева

Каждая из трёх специализированных панелей хранит собственный стек истории состояний (атрибут `history`), реализованный как список глубоких копий модели, сделанных

непосредственно перед выполнением каждой изменяющей операции. Эта же техника лежит в основе функции Undo: при её вызове текущее состояние модели заменяется последним сохранённым состоянием из истории, после чего производится повторная отрисовка холста.

2.4 Выбор средств тестирования и контроля корректности

Поскольку приложение носит интерактивный характер и взаимодействует с пользователем через графический интерфейс, для контроля корректности реализованных алгоритмов был выбран комбинированный подход: модульная логика структур данных проверялась независимо от графического интерфейса путём непосредственного вызова их методов с контрольными наборами значений и сравнением результата с ожидаемым; корректность же визуального отображения и реакции интерфейса на действия пользователя проверялась посредством автоматизированного управления запущенным экземпляром приложения с фиксацией состояния холста на каждом шаге выполнения сценария [13].

Такой комбинированный подход обеспечивает всестороннюю проверку приложения на двух ключевых уровнях. Во-первых, модульное тестирование алгоритмов структур данных в изоляции от графического интерфейса позволяет гарантировать их корректность независимо от визуальной составляющей, упрощает локализацию логических ошибок и даёт возможность применять стандартные средства автоматического модульного тестирования [14]. Во-вторых, автоматизированное управление пользовательским интерфейсом с фиксацией состояния холста на каждом шаге сценария подтверждает, что графическое представление точно соответствует внутреннему состоянию структуры, а все элементы взаимодействия реагируют предсказуемо. Фиксация визуального состояния делает возможным регрессионное тестирование отображения – сравнение текущих изображений с эталонными и своевременное выявление непреднамеренных изменений в визуализации [15]. Совместное применение модульной проверки логики и сквозного тестирования интерфейса создаёт надёжный барьер для дефектов как в алгоритмической части, так и в её графическом воплощении, что в итоге повышает уверенность в работоспособности приложения.

Важно отметить, что раздельное тестирование модельного слоя и слоя представления соответствует общепринятой практике разработки программного обеспечения, при которой логика, не зависящая от пользовательского интерфейса, выносится в отдельно проверяемые модули: это не только упрощает обнаружение ошибок на ранних этапах разработки, но и делает код более пригодным для повторного использования и дальнейшего расширения функциональности приложения.

3 Реализация и тестирование визуализатора структур данных

3.1 Общая структура программы и точка входа

Приложение реализовано в виде единого исполняемого модуля на языке Python 3 с использованием стандартной библиотеки `tkinter` и её расширения `ttk` (предоставляющего стилизованные виджеты, в частности компонент `Notebook` для организации вкладок). Главный класс приложения `App` наследуется от `tk.Tk` и при инициализации создаёт три вкладки – для стека, очереди и дерева, – а также единую для всех трёх структур текстовую область журнала операций, расположенную в нижней части окна.

Метод `mainloop()` запускает основной цикл обработки событий операционной системы (нажатия клавиш, движения мыши, события перерисовки окна), являющийся неотъемлемой частью любого GUI-приложения на `Tkinter` и одинаково функционирующий как под управлением `Windows`, так и под управлением `Linux`, поскольку соответствующий цикл реализован внутри библиотеки `Tcl/Tk`, а не средствами конкретной ОС.

Логика стека инкапсулирована в классе `Stack`, использующем внутренний список Python в качестве хранилища элементов. Операция `push` реализована через метод `list.append`, добавляющий элемент в конец внутреннего списка (что соответствует логической вершине стека), а операция `pop` – через `list.pop()` без аргумента, удаляющий и возвращающий последний элемент списка. Такая реализация гарантирует выполнение операций `push`, `pop` и `peek` за время $O(1)$, поскольку добавление и удаление элемента в конце динамического массива Python в среднем не требует перераспределения памяти и сдвига остальных элементов. Защита от ошибки извлечения из пустого стека реализована на уровне самого метода (выражение-условие `if self._data else None` возвращает `None` при пустом стеке вместо возбуждения исключения `IndexError`), что упрощает обработку этого случая на уровне графического интерфейса [16].

3.2 Панель управления стеком и отображение состояния

Класс `StackPanel` связывает экземпляр `Stack` с виджетом `Canvas` и набором из четырёх кнопок управления: `Push`, `Pop`, `Peek` и `Undo`. Метод `_draw`, вызываемый после выполнения каждой операции, полностью очищает содержимое холста (`canvas.delete("all")`) и заново отрисовывает все элементы стека в виде вертикальной башни прямоугольников, выстроенной от нижнего края холста вверх – таким образом, что верхний прямоугольник соответствует элементу, к которому применима следующая операция `pop` или `peek`. На рисунке 4 показано состояние интерфейса вкладки «Стек» непосредственно после трёх последовательных операций `push` со значениями 10, 20 и 30.

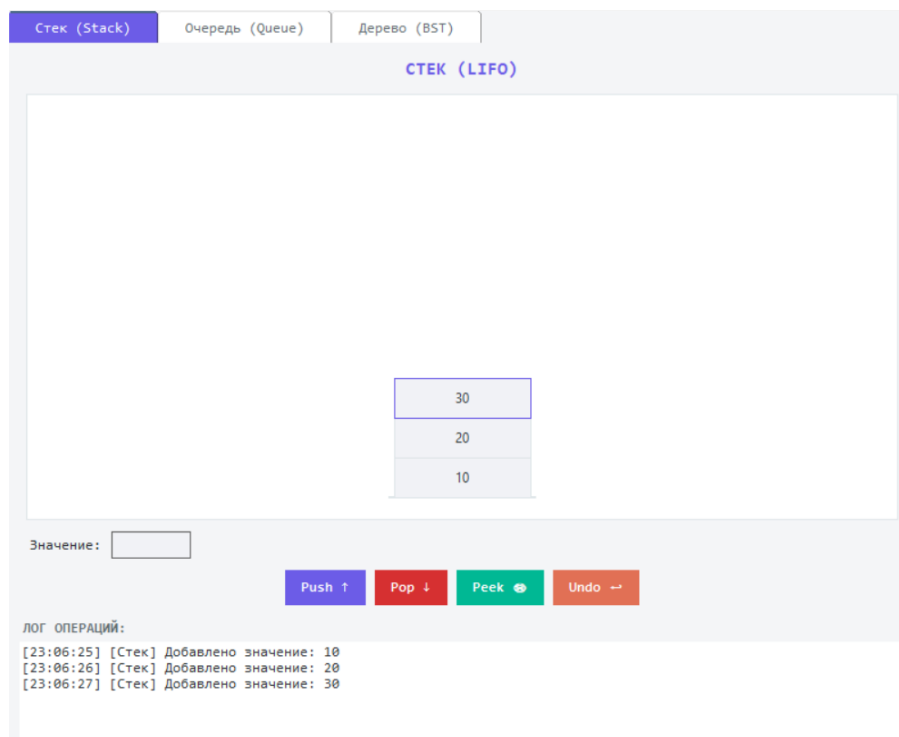


Рисунок 4 – Состояние стека после операций push(10), push(20), push(30)

При вызове операции peek верхний элемент стека выделяется отдельным цветом заливки (параметр hl=True метода `_draw`), что наглядно демонстрирует, какой именно элемент возвращается операцией без его извлечения из структуры. Результат вызова peek для состояния стека, показанного на рисунке 4, приведён на рисунке 5.

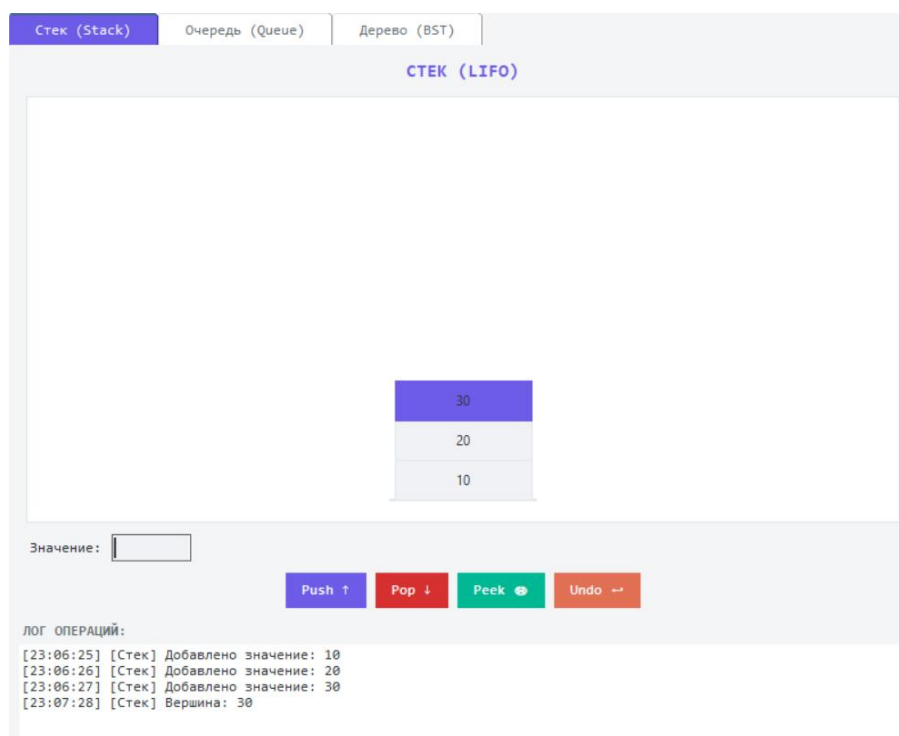


Рисунок 5 – Подсветка верхнего элемента стека при выполнении операции peek

После выполнения операции pop верхний элемент удаляется из внутреннего представления и, соответственно, из отрисованной башни – высота столбца уменьшается на один прямоугольник, что отражено на рисунке 6.

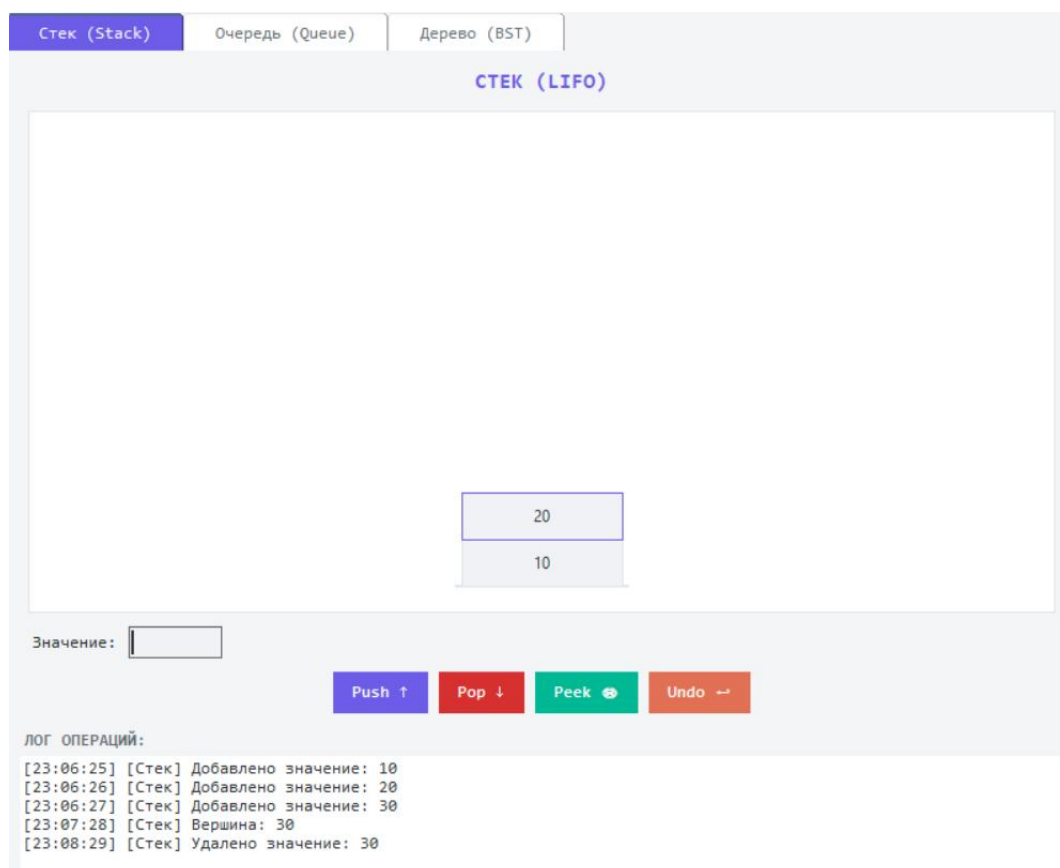


Рисунок 6 – Состояние стека после выполнения операции pop (верхний элемент со значением 30 удалён)

Разработанный интерфейс обеспечивает прямое взаимодействие с моделью стека, позволяя в реальном времени отслеживать текущее состояние структуры и результаты выполнения методов push, pop и peek.

3.3 Механизм Undo для стека

Перед выполнением каждой изменяющей состояние операции (push, pop) текущее состояние стека сохраняется в список self.history путём вызова метода copy(), создающего независимую копию внутреннего списка значений. Использование копии, а не прямой ссылки на исходный список, является важным техническим решением: поскольку списки в Python являются изменяемым типом данных, сохранение ссылки вместо копии привело бы к тому, что все элементы истории указывали бы на один и тот же объект в памяти, и любое изменение текущего состояния стека автоматически отражалось бы на всех ранее сохранённых снимках, делая механизм отмены операций неработоспособным. Такой подход к организации истории изменений близок к паттерну проектирования «Снимок», широко применяемому для

реализации отмены действий в текстовых и графических редакторах и других интерактивных приложениях [17]. При нажатии кнопки Undo последний сохранённый снимок состояния извлекается из history и становится текущим состоянием стека, после чего выполняется перерисовка холста, обеспечивающая визуальное отражение восстановленного состояния для пользователя. На рисунке 7 показан результат отмены последней операции pop, выполненной для иллюстрации на рисунке 6 – стек возвращается к виду, идентичному рисунку 4.

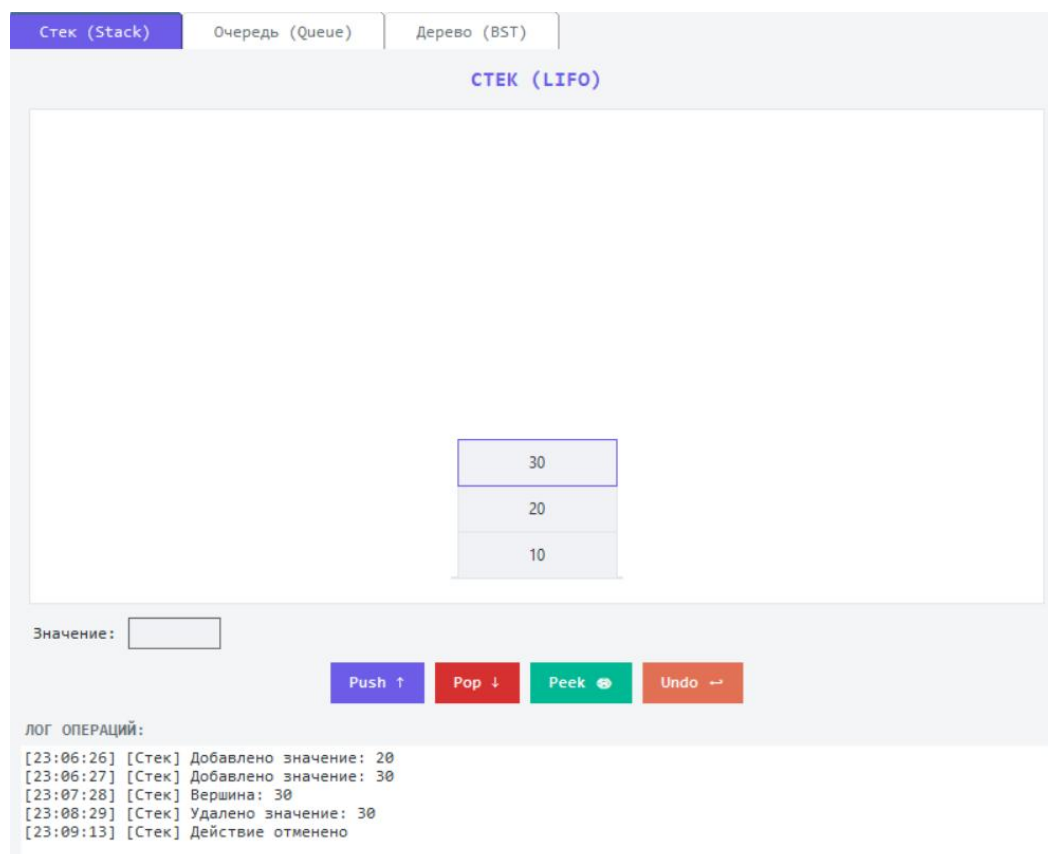


Рисунок 7 – Восстановление состояния стека после операции Undo (отменена операция pop)

Представленный механизм сохранения состояний дает возможность корректно возвращаться к предыдущим этапам работы со стеком, гарантируя целостность данных при отмене последних действий пользователя.

3.4 Реализация и визуализация очереди

3.4.1 Класс Queue и его операции

Класс Queue по структуре аналогичен классу Stack, однако реализует операции в соответствии с дисциплиной FIFO. Операция enqueue, как и push, добавляет элемент в конец внутреннего списка методом append, тогда как операция dequeue, в отличие от pop, извлекает элемент с начала списка методом pop(0)

Необходимо отметить, что вызов list.pop(0) для динамического массива Python требует сдвига всех оставшихся элементов на одну позицию влево и потому выполняется за время

$O(n)$, а не $O(1)$, как в случае стека. Для целей учебной визуализации, где число одновременно отображаемых элементов очереди ограничено размерами холста (как правило, не более 10–15 элементов), эта особенность не оказывает заметного влияния на отзывчивость интерфейса; при необходимости масштабирования решения для больших объёмов данных внутреннее представление очереди целесообразно заменить на структуру `collections.deque` из стандартной библиотеки Python3, обеспечивающую амортизированную сложность $O(1)$ для операций добавления и извлечения с обоих концов.

3.4.2 Панель управления очередью и отображение состояния

Класс `QueuePanel` непосредственно отрисовывает все текущие элементы исследуемой очереди в виде последовательного горизонтального ряда прямоугольников строго слева направо, с соответствующими текстовыми подписями «front» над первым элементом и «back» под последним из них, что наглядно и максимально прозрачно показывает пользователю общее направление движения элементов внутри структуры: новые значения добавляются справа (со стороны подписи «back»), а извлекаются слева (со стороны подписи «front»). Текущее визуальное состояние очереди после успешного выполнения четырёх последовательных операций добавления `enqueue` со значениями 5, 15, 25 и 35 детально показано на приведённом ниже рисунке 8.

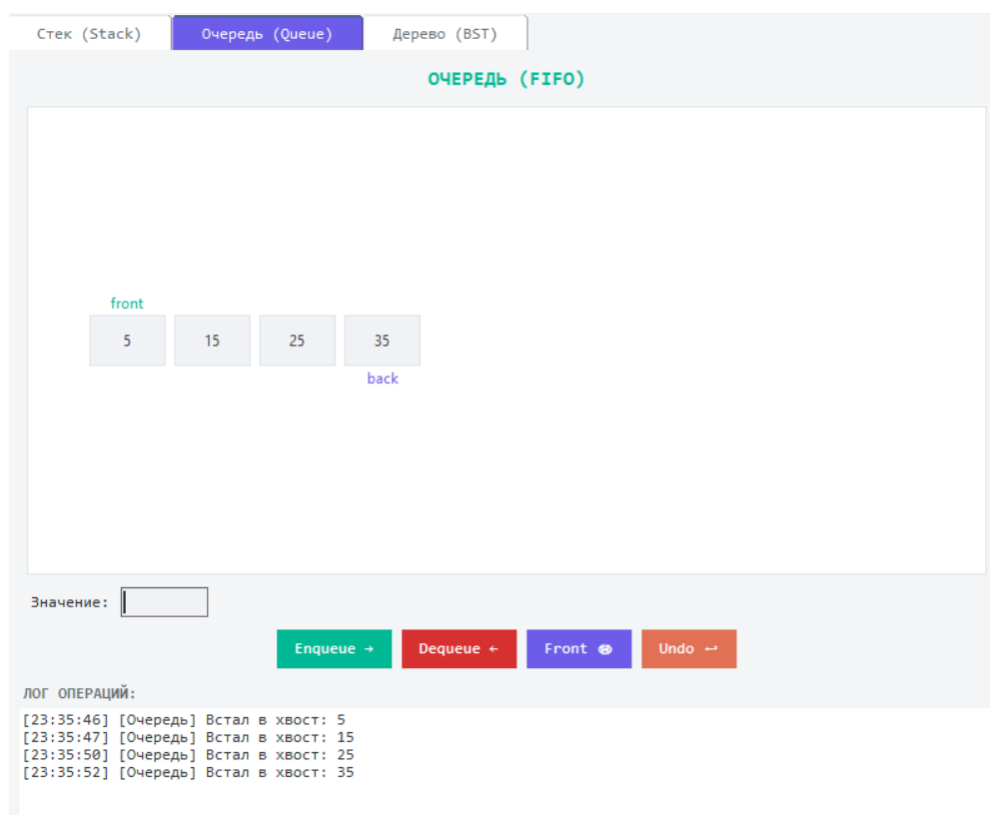


Рисунок 8 – Состояние очереди после операций `enqueue`, `enqueue`, `enqueue`, `enqueue`

Вызываемая в графическом интерфейсе операция `front`, аналогично стандартному методу `peek` для стека, визуально выделяет первый элемент текущей очереди контрастной цветовой подсветкой без его непосредственного извлечения (рисунок 9).

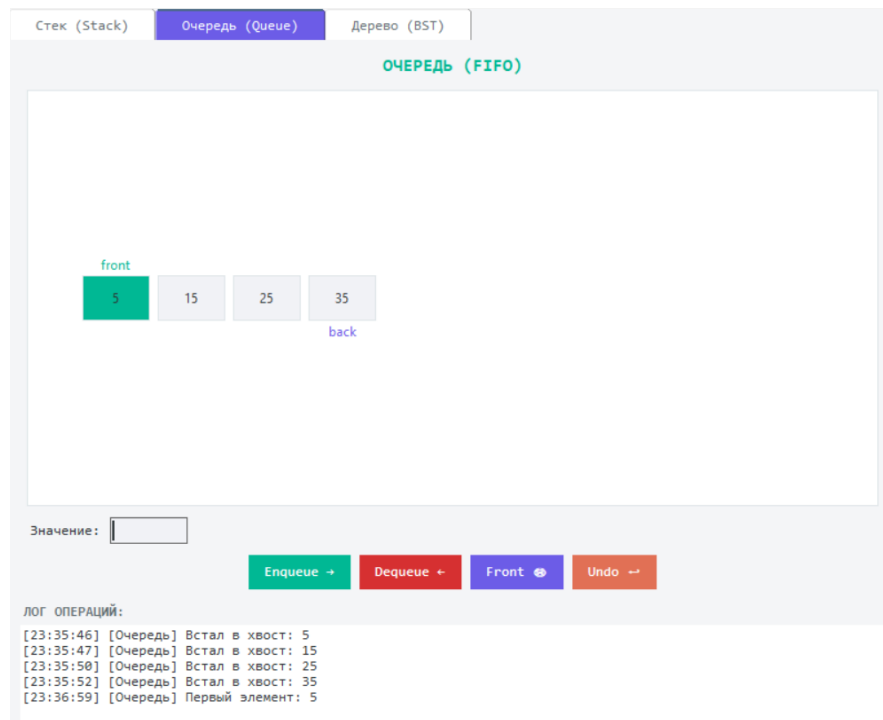


Рисунок 9 – Подсветка первого элемента очереди при выполнении операции `front`

Операция `dequeue` удаляет этот элемент из структуры, сдвигая визуальное представление оставшихся элементов (рисунок 10).

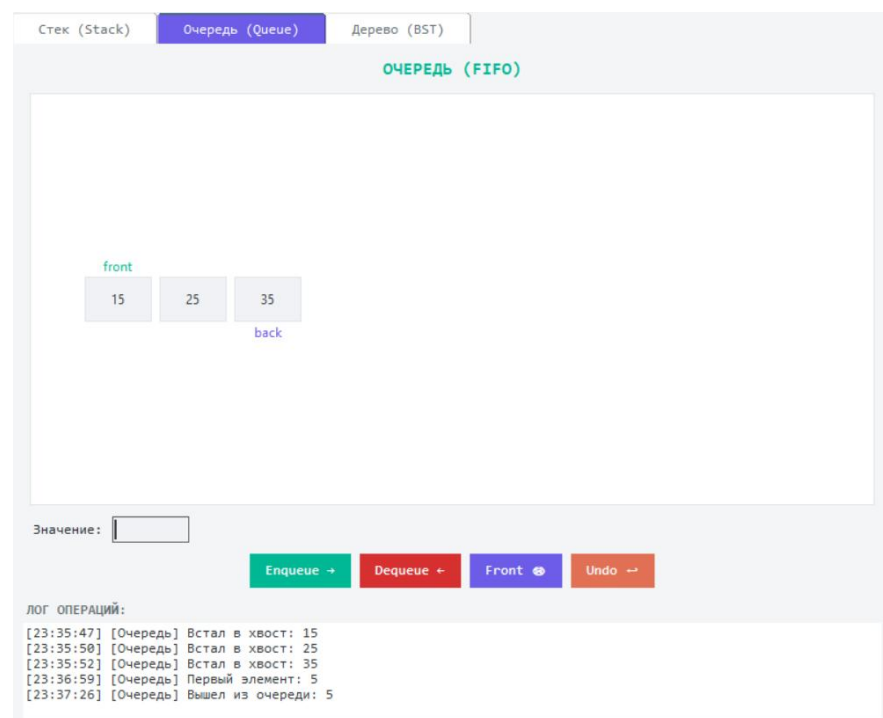


Рисунок 10 – Состояние очереди после выполнения операции `dequeue` (элемент со значением 5 извлечён из начала очереди)

Графические элементы управления корректно визуализируют принцип FIFO, синхронно обновляя отображение модели при добавлении (enqueue) или извлечении (dequeue) элементов из очереди.

3.4.3 Механизм Undo для очереди

Механизм отмены действия для очереди реализован полностью аналогично механизму, описанному в подразделе 3.3 для стека: перед каждой изменяющей операцией сохраняется копия состояния, а вызов Undo восстанавливает предыдущий сохранённый снимок. Идентичность реализации данного механизма для всех трёх структур данных, рассматриваемых в настоящей работе, подтверждает универсальность выбранного архитектурного решения и упрощает сопровождение кода в дальнейшем – добавление аналогичной функциональности для новой структуры данных, не предусмотренной текущим заданием, потребовало бы лишь реализации метода `copy()` для её класса модели.

3.5 Реализация и визуализация двоичного дерева поиска

Узел дерева представлен минималистичным классом `BSTNode`, хранящим значение узла и ссылки на левого и правого потомка (`None` при отсутствии соответствующего потомка). Класс `BST` хранит ссылку на корневой узел и реализует три основные операции рекурсивно – через приватные вспомогательные методы с префиксом подчёркивания (`_insert`, `_delete`), принимающие текущий узел поддеревы и возвращающие обновлённую ссылку на него, что является типичным идиоматическим приёмом реализации рекурсивных операций над деревьями на языке Python.

Операция поиска реализована итеративно (без рекурсии) и одновременно накапливает список пройденных значений узлов в атрибуте `last_path`, который впоследствии используется слоем визуализации для подсветки пути поиска.

Наиболее алгоритмически интересным фрагментом реализации является обработка случая удаления узла, имеющего двух потомков. В соответствии с теоретическим описанием, приведённым в подразделе 1.4, узел заменяется минимальным значением своего правого поддеревы, после чего это значение рекурсивно удаляется из правого поддеревы.

Корректность данного фрагмента подтверждена практической демонстрацией на контрольном дереве, построенном вставкой значений 50, 30, 70, 20, 40, 60 и 80 в указанном порядке. Структура построенного таким образом дерева показана на рисунке 11: значение 50 образует корень, в его левом поддереве располагаются значения, меньшие 50 (30, 20, 40), а в правом – значения, большие 50 (70, 60, 80), что в точности соответствует основному инварианту BST, рассмотренному в подразделе 1.4.

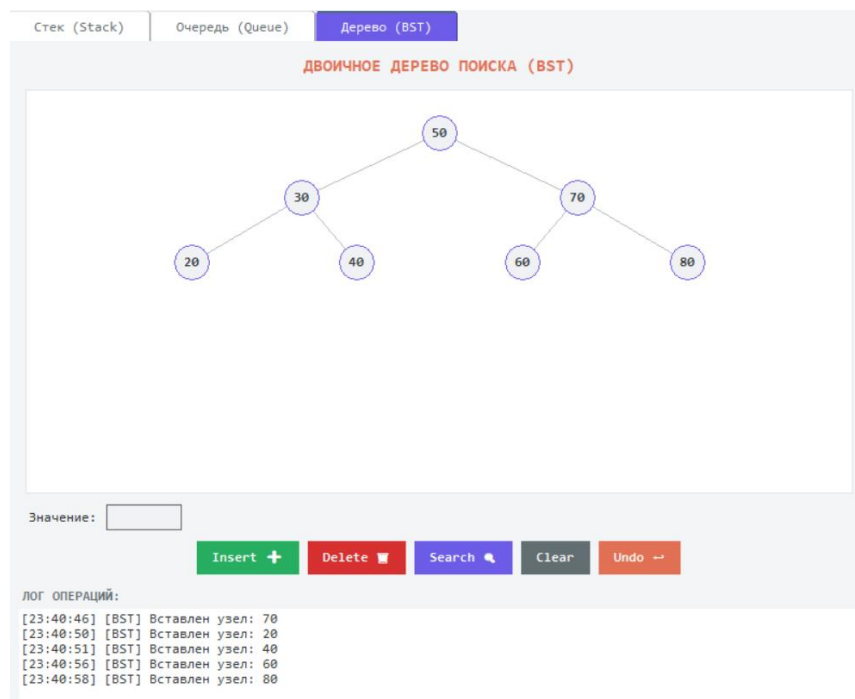


Рисунок 11 – Контрольное дерево после последовательной вставки значений

При выполнении операции search со значением 40 на построенном дереве путь поиска проходит через узлы 50 и 30 перед обнаружением искомого значения. Узлы пути (за исключением целевого) подсвечиваются на холсте жёлтым цветом, а сам найденный узел – зелёным; одновременно строка пути добавляется в текстовый журнал операций в виде списка [50, 30, 40]. Такая цветовая индикация и логирование позволяют пользователю наглядно проследить каждый шаг алгоритма и сопоставить его с теоретическим описанием. Результат данной операции показан на рисунке 12.

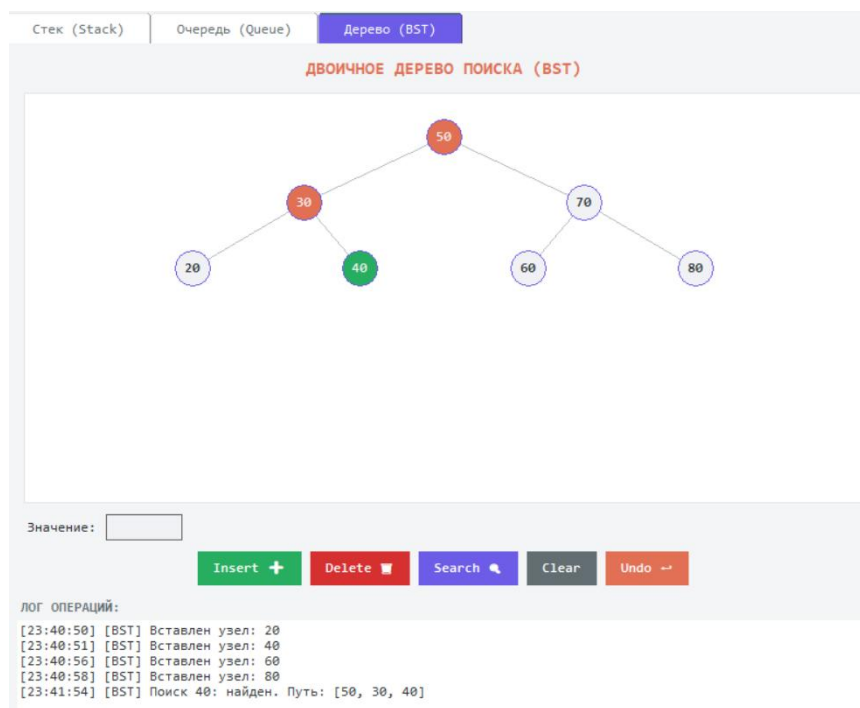


Рисунок 12 – Визуализация пути поиска значения 40: путь [50, 30, 40]

В том конкретном случае, если определённое искомое значение в текущем дереве полностью отсутствует, весь путь поиска автоматически прерывается на последнем посещённом узле, после которого дальнейшее требуемое движение в пустое поддереву становится невозможно; в рабочий журнал программы сразу же заносится текстовая запись с указанием пройденного пути и финального статуса «не найден», как достаточно наглядно показано на приведённом рисунке 13.

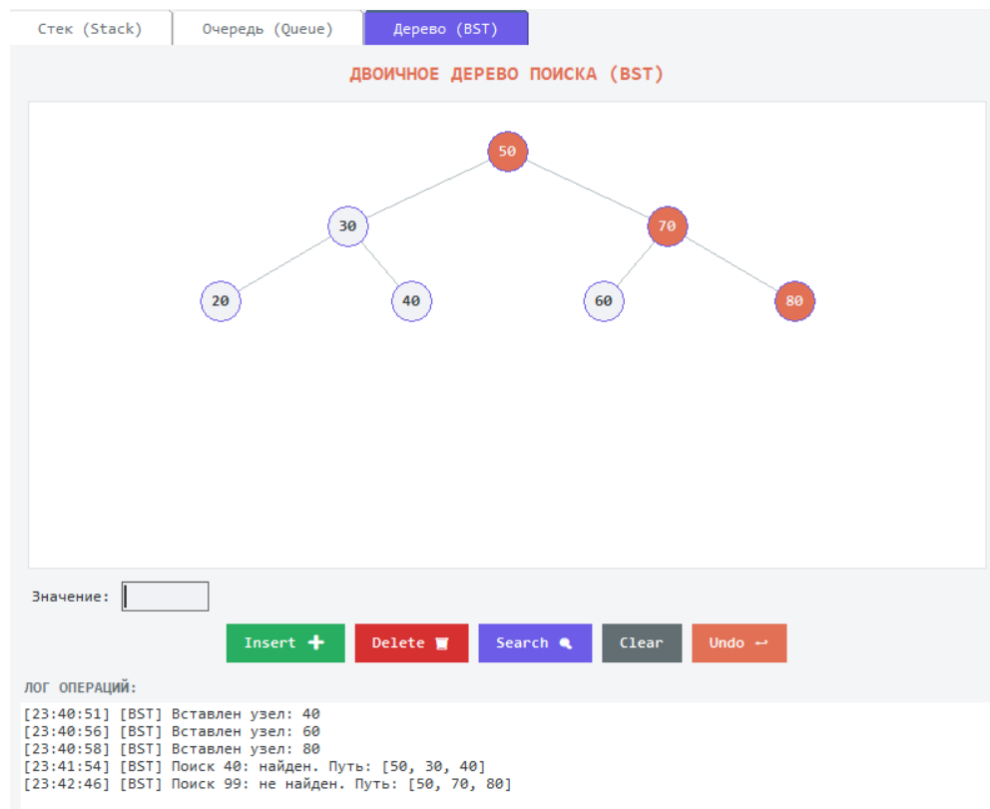


Рисунок 13 – Результат поиска отсутствующего в дереве значения 99: путь [50, 70, 80]

Выполняемое на данном практическом этапе удаление узла со значением 30, изначально имеющего двух непосредственных потомков (20 и 40), весьма наглядно демонстрирует работу алгоритма, ранее подробно описанного в самом начале данного подраздела: узел 30 автоматически замещается значением 40 (минимальным значением своего правого поддерева, в данном конкретном случае состоящего лишь из единственного узла), а исходный узел 40 безвозвратно удаляется из структуры дерева как лист. Сформированная итоговая структура бинарного дерева после данной операции показана на рисунке 14 – узел со значением 40 теперь является прямым потомком корня, успешно замещая удалённый узел 30, при этом узел 20 остался на своём законном месте в качестве левого потомка нового элемента 40.

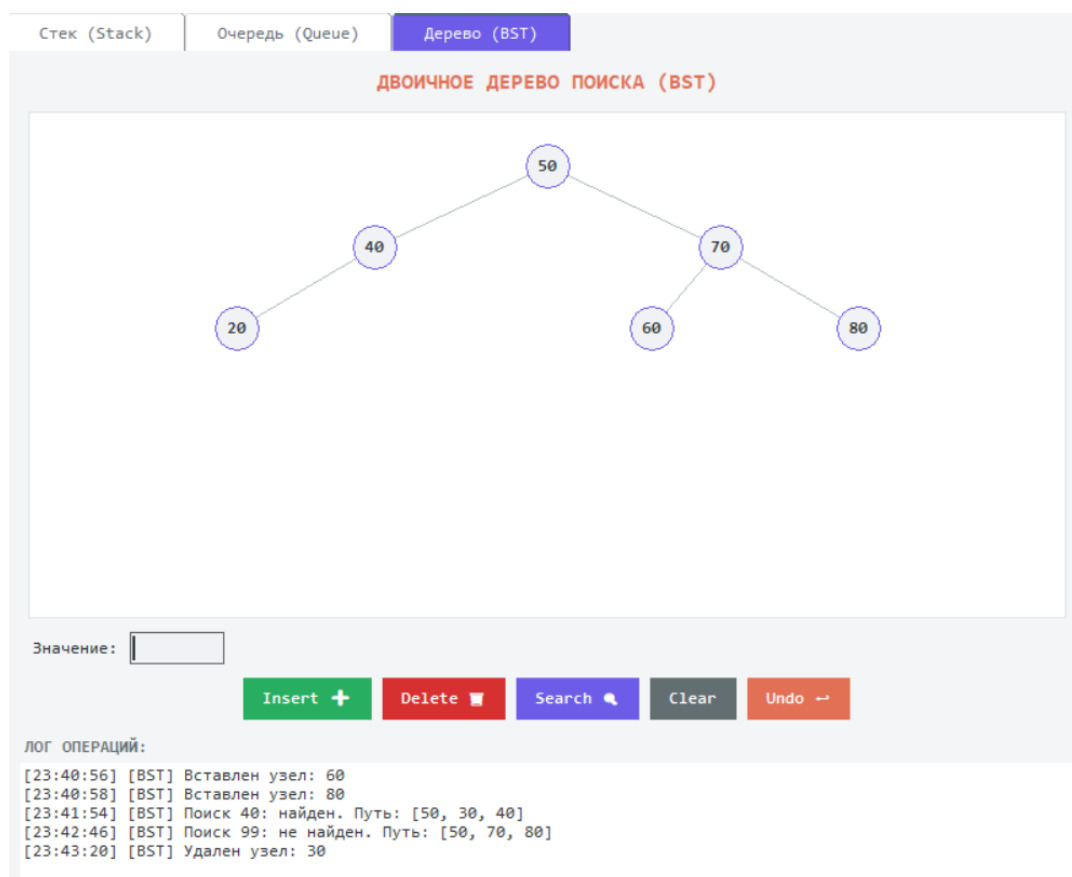


Рисунок 14 – Состояние дерева после удаления узла со значением 30: узел замещён значением 40 из правого поддерева

Графический вывод бинарного дерева поиска перестраивается при вставке и удалении узлов, обеспечивая правильное отображение иерархии потомков и наглядную индикацию результатов поиска по ключу.

Поскольку Tkinter не предоставляет встроенного механизма автоматической раскладки деревьев, в приложении реализован собственный рекурсивный алгоритм вычисления координат узлов `_calc_positions`, основанный на представлении дерева в виде неявной нумерации полного двоичного дерева (аналогичной нумерации узлов в пирамиде, она же `binary heap`): корень получает условный индекс 1, левый потомок узла с индексом i получает индекс $2i-1$, а правый – индекс $2i$.

Текущая горизонтальная координата каждого отдельного узла вычисляется строго пропорционально его условному индексу относительно удвоенного на каждом последующем уровне общего числа возможных позиций (параметр `total`), что в конечном итоге обеспечивает максимально равномерное распределение узлов по всей доступной ширине холста без возникновения нежелательных пересечений линий связи даже для сложных несимметричных деревьев, а соответствующая вертикальная координата заданного элемента гарантированно определяется глубиной нахождения узла (`level`), умноженной на определённый фиксированный шаг между соседними уровнями визуализации.

3.6 Тестирование корректности реализованных алгоритмов

Тестирование разработанного приложения проводилось по двум направлениям: проверка корректности самой логики структур данных на контрольных наборах значений и проверка работоспособности графического интерфейса и его реакции на пользовательские действия путём автоматизированного сценарного прогона приложения с фиксацией состояния холста после каждого шага. В рамках первого направления на вход алгоритмам подавались эталонные последовательности операций, включая вырожденные и граничные случаи (пустые структуры, многократные вставки и удаления), после чего результаты сравнивались с ожидаемыми. Для графической части были подготовлены сценарии, воспроизводящие типичные действия пользователя: последовательное добавление, перемещение и удаление узлов, а также изменение параметров визуализации. Сохранённые скриншоты холста после каждого шага позволили отследить корректность отрисовки, отсутствие артефактов и своевременное обновление изображения при любых манипуляциях.

Для стека выполнен следующий контрольный сценарий: последовательные операции `push` со значениями 10, 20 и 30, затем операция `peek` (ожидаемый результат – 30, верхний элемент), затем операция `pop` (ожидаемый результат – извлечено значение 30, в стеке остаются 10 и 20), затем операция `Undo` (ожидаемый результат – восстановление стека к состоянию [10, 20, 30]). Все шаги сценария дали ожидаемый результат, что подтверждается последовательностью иллюстраций на рисунках 4–7 настоящего отчёта.

Для очереди выполнен аналогичный сценарий: операции `enqueue` со значениями 5, 15, 25 и 35 (с проверкой того, что новые элементы добавляются справа, в конец очереди), операция `front` (ожидаемый результат – 5, первый добавленный элемент), операция `dequeue` (ожидаемый результат – извлечено значение 5, в очереди остаются 15, 25, 35 – то есть проверена корректность дисциплины FIFO в противовес дисциплине LIFO стека), и операция `Undo`. Результаты данного сценария отражены на рисунках 8–10.

Для дерева поиска выполнен наиболее развёрнутый сценарий, включающий построение контрольного дерева из семи значений (50, 30, 70, 20, 40, 60, 80), поиск присутствующего значения 40 (ожидаемый результат – найдено, путь [50, 30, 40]), поиск отсутствующего значения 99 (ожидаемый результат – не найдено, путь [50, 70, 80]) и удаление узла 30 с двумя потомками (ожидаемый результат – узел замещён значением 40 из правого поддерева в соответствии с алгоритмом, описанным в подразделе 3.4.2). Дополнительно проверена операция `Undo`, корректно восстанавливающая дерево к состоянию, предшествовавшему удалению. Результаты данного сценария отражены на рисунках 11–14. Сводная таблица контрольных сценариев тестирования и их результатов представлена в таблице 3.

Таблица 3 – Сводные результаты тестирования структур данных

Структура	Сценарий	Ожидаемый результат	Статус
Стек	push(10,20,30) → peek → pop → undo	Вершина 30, после pop вершина 20, после undo вершина 30	пройден
Очередь	enqueue(5,15,25,35) → front → dequeue → undo	Начало очереди 5, после dequeue начало 15, после undo начало 5	пройден
BST	insert(50,30,70,20,40,60,80) → search(40)	Найдено, путь [50, 30, 40]	пройден
BST	search(99) на том же дереве	Не найдено, путь [50, 70, 80]	пройден
BST	delete(30) → undo	Узел 30 замещён значением 40; undo восстанавливает узел 30	пройден

Анализ данных, приведённых в таблице 3, позволяет сделать вывод о корректной реализации базовых операций визуализатора: вставка, поиск и удаление выполняются в соответствии с логикой бинарного дерева поиска, а механизм Undo обеспечивает надёжный возврат к предыдущим состояниям.

3.7 Проверка кроссплатформенной работоспособности

Помимо функционального тестирования логики структур данных, отдельное внимание было уделено проверке работоспособности приложения в среде, отличной от типичной локальной среды разработки Windows. Это требование особенно актуально для университетской среды, где в учебных аудиториях и лабораториях могут быть установлены как Windows, так и различные дистрибутивы Linux. Собранный средствами PyInstaller [18] автономный исполняемый файл был запущен в среде Linux (Ubuntu). Приложение запустилось без ошибок, что служит прямым практическим подтверждением кроссплатформенности разработанного решения, обоснованной теоретически в подразделе 2.2. Фактический запуск и корректная визуальная работа автономного исполняемого файла в среде Linux являются эмпирическим доказательством того же тезиса, который для среды Windows обеспечивается идентичным способом сборки. Поскольку PyInstaller упаковывает все необходимые библиотеки и зависимости, включая привязки Tcl/Tk, непосредственно в исполняемый файл, приложение не требует от пользователя установки интерпретатора Python или каких-либо дополнительных системных пакетов, что делает развёртывание программы на обеих платформах тривиальным и не создаёт препятствий для конечного пользователя.

Заключение

В ходе прохождения учебной ознакомительной практики выполнены все задачи, поставленные индивидуальным заданием. Изучены теоретические основы трёх классических структур данных – стека, очереди и двоичного дерева поиска, – включая принципы их работы. Проведён сравнительный анализ доступных средств реализации кроссплатформенного графического интерфейса на языке Python, по результатам которого обоснованно выбрана стандартная библиотека Tkinter как решение, обеспечивающее переносимость между операционными системами Windows и Linux без дополнительных внешних зависимостей.

Спроектирована и реализована архитектура приложения, разделяющая логику структур данных и компоненты графического представления, что обеспечило единообразие реализации механизма пошагового выполнения операций, динамической перерисовки состояния структур на холсте и ведения общего журнала операций для всех трёх структур данных одновременно.

Корректность работы реализованных алгоритмов подтверждена тестированием на контрольных наборах данных: для стека и очереди проверена корректность дисциплин LIFO и FIFO соответственно, для двоичного дерева поиска дополнительно проверена корректность операции поиска с визуализацией пройденного пути, а также корректность алгоритмически нетривиальной операции удаления узла, имеющего двух потомков.

Заявленное требование кроссплатформенности выполнено и подтверждено не только теоретическим обоснованием выбора технологического стека, но и практической проверкой: приложение было запущено и протестировано непосредственно в среде Linux, при этом все продемонстрированные в отчёте иллюстрации работы интерфейса получены в результате такого запуска, без внесения каких-либо изменений в исходный код приложения по сравнению с его эталонной версией, разработанной для среды Windows. Все необходимые для ознакомления материалы проекта, включая исходный код разработанного программного обеспечения и автономные исполняемые файлы для целевых операционных систем Windows (EXE) и Linux (ELF), были в полном объёме выгружены в удаленный репозиторий на платформе GitHub по ссылке: <https://github.com/Alelovv/DataStructures>.

Таким образом, цель работы – разработка кроссплатформенного программного обеспечения на языке Python для интерактивной визуализации стека, очереди и двоичного дерева поиска с поддержкой пошагового выполнения операций, динамического отображения состояний структур и ведения лога с функцией отмены действий – достигнута полностью. Результаты практики могут быть использованы в дальнейшем как основа для расширения функциональности визуализатора либо как самостоятельный учебно-демонстрационный инструмент при изучении дисциплин, связанных со структурами данных и алгоритмами.

Список использованных источников

- 1 СК-СТО-ТР-04-1.005-2015. Требования к оформлению текстовой части выпускных квалификационных работ, курсовых работ (проектов), рефератов, контрольных работ, отчетов по практикам, лабораторным работам. – Владивосток: ВВГУ, 2015.
- 2 Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн ; пер. с англ. – 3-е изд. – Москва : Вильямс, 2013. – 1328 с.
- 3 ГОСТ 19.701–90 (ИСО 5807–85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. – Москва : Стандартинформ, 2010. – 26 с.
- 4 Модуль collections [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/collections.html> (дата обращения: 08.06.2026).
- 5 Документация Java SE [Электронный ресурс]. – Режим доступа: <https://docs.oracle.com/en/java/javase/> (дата обращения: 09.06.2026).
- 6 Документация Qt 6 [Электронный ресурс]. – Режим доступа: <https://doc.qt.io/qt-6/> (дата обращения: 09.06.2026).
- 7 Документация .NET MAUI [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/dotnet/maui/> (дата обращения: 09.06.2026).
- 8 Модуль tkinter [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 10.06.2026).
- 9 Документация Qt for Python (PySide6) [Электронный ресурс]. – Режим доступа: <https://doc.qt.io/qtforpython/> (дата обращения: 10.06.2026).
- 10 Документация Flask [Электронный ресурс]. – Режим доступа: <https://flask.palletsprojects.com/> (дата обращения: 10.06.2026).
- 11 Документация Django [Электронный ресурс]. – Режим доступа: <https://docs.djangoproject.com/> (дата обращения: 10.06.2026).
- 12 Ларман, К. Применение UML 2.0 и шаблонов проектирования : практическое руководство / К. Ларман ; пер. с англ. – 3-е изд. – Москва : Вильямс, 2013. – 736 с.
- 13 Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер ; пер. с англ. – 3-е изд. – Москва : Диалектика, 2019. – 272 с.
- 14 Модуль unittest [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/unittest.html> (дата обращения: 14.06.2026).
- 15 Документация библиотеки Pillow (PIL Fork) [Электронный ресурс]. – Режим доступа: <https://pillow.readthedocs.io/> (дата обращения: 14.06.2026).
- 16 Лутц, М. Изучаем Python : в 2 т. Т. 1 / М. Лутц ; пер. с англ. – 5-е изд. – Санкт-Петербур

г : Питер, 2019. – 832 с.

17 Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес ; пер. с англ. – Санкт-Петербург : Питер, 2020. – 366 с.

18 Документация PyInstaller [Электронный ресурс]. – Режим доступа: <https://pyinstaller.org/en/stable/> (дата обращения: 18.06.2026).

Приложение А

Блок-схема алгоритма двоичного дерева поиска (BST)

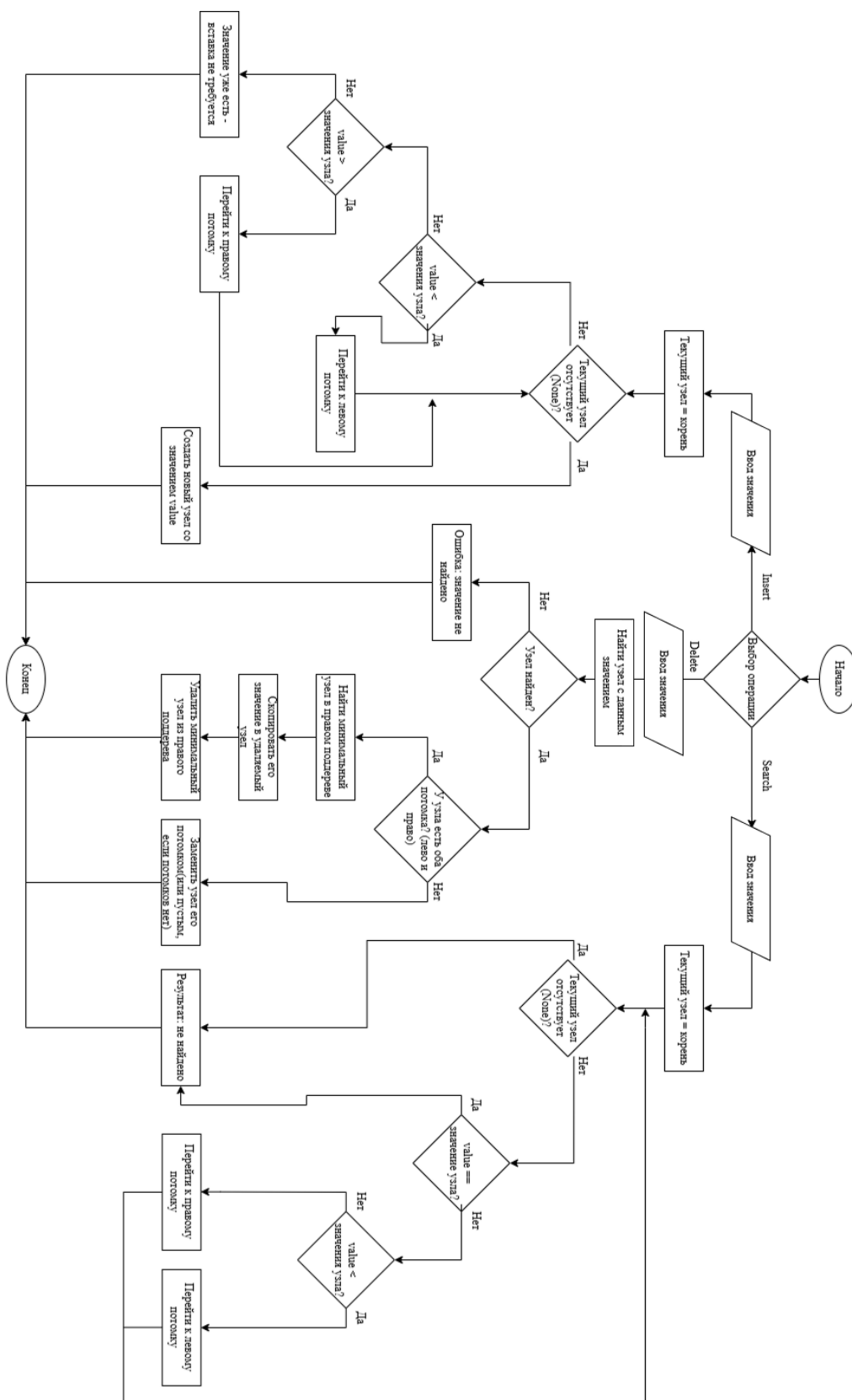


Рисунок А.1 – Блок-схема алгоритма двоичного дерева поиска (BST)